

1. Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

Run SQL query/queries on table restaurant.intluencers: ?

```
1 SELECT user_id, COUNT(*) AS order_count
2 FROM orders
3 GROUP BY user_id;
```

Run SQL query/queries on database restaurant: ?

```
1 CREATE TABLE orders (
2     order_id INT PRIMARY KEY,
3     user_id INT
4 );
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

order_id user_id

Query results operations

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#)

Run SQL query/queries on table restaurant.orders: ?

```
1 INSERT INTO orders VALUES
2 (1, 101),
3 (2, 101),
4 (3, 102),
5 (4, 103),
6 (5, 101),
7 (6, 102);
8 SELECT user_id, COUNT(*) AS order_count
9 FROM orders
10 GROUP BY user_id;
```

☐ Show all	Number of rows: 25
-----------------------------------	--------------------

Extra options

user_id	order_count
101	3
102	2
103	1

2. Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

Browse
Structure
SQL
Search
Insert
Export
Import

Run SQL query/queries on table restaurant.restaurants: ?

```

1 SELECT name, AVG(rating) AS average_rating
2 FROM restaurants
3 GROUP BY name
4 HAVING AVG(rating) > 4.0;

```


☐ Show all	Number of rows: 25	Filter rows: Search this table
-----------------------------------	--------------------	--------------------------------

Extra options

	name	average_rating
☐ Edit Copy Delete	Pizza Hub	4.20000
☐ Edit Copy Delete	Spice Villa	4.70000
☐ Edit Copy Delete	Sushi World	4.60000

☐ Check all
With selected: Edit Copy Delete Export

3. Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.

Run SQL query/queries on database restaurant: 

```
1 CREATE TABLE payments (  
2     user_id INT,  
3     amount DECIMAL(10,2),  
4     payment_date DATE  
5 );  
6  
7 INSERT INTO payments VALUES  
8 (101, 1500, '2026-09-01'),  
9 (102, 2500, '2026-09-02'),  
10 (103, 1800, '2026-09-03'),  
11 (101, 3000, '2026-09-04'),  
12 (104, 1200, '2026-09-05');  
13  
14 SELECT user_id  
15 FROM payments
```

Extra options

user_id	amount	payment_date
101	1500.00	2026-09-01
102	2500.00	2026-09-02
103	1800.00	2026-09-03
101	3000.00	2026-09-04
104	1200.00	2026-09-05

☐ Show all | Number of rows: 25  Filter rows:

Extra options

user_id
101
102
101

4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.

Run SQL query/queries on database restaurant: ?

```
1 CREATE TABLE movies (  
2     movie_id INT PRIMARY KEY,  
3     movie_name VARCHAR(100),  
4     rating DECIMAL(3,1)  
5 );  
6  
7 INSERT INTO movies VALUES  
8 (1, 'Inception', 4.5),  
9 (2, 'Avatar', 4.0),  
10 (3, 'Interstellar', 4.8),  
11 (4, 'Titanic', 4.2),  
12 (5, 'Joker', 3.8);  
13  
14 SELECT movie_name, AVG(rating) AS average_rating  
15 FROM movies
```

Extra options

				movie_id	movie_name	rating
☐		Edit		Copy		Delete
				1	Inception	4.5
☐		Edit		Copy		Delete
				2	Avatar	4.0
☐		Edit		Copy		Delete
				3	Interstellar	4.8
☐		Edit		Copy		Delete
				4	Titanic	4.2
☐		Edit		Copy		Delete
				5	Joker	3.8

 ☐ Check all With selected:  Edit  Copy  Delete  Export

☐ Show all

Number of rows: 25

Filter rows:

Extra options

				movie_name	average_rating
☐	 Edit	 Copy	 Delete	Inception	4.50000
☐	 Edit	 Copy	 Delete	Interstellar	4.80000

 ☐ Check all

With selected:

 Edit

 Copy

 Delete